

סיכום תרגולים 2026 – מערכות הפעלה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

הערה. לפי המרצה, התרגולים מכסים לרוב חומר שונה לגמרי מהחומר בהרצאה ואף נשאלים במבחן בחלק ייעודי שנקרא חומר מהתרגול, לכן הסיכום הנ"ל מפוצל מסיכום ההרצאה.

תוכן עניינים:

2	לינוקס, system calls, תרגול 1: מבוא למערכות הפעלה
5	bash, תרגול 2:
8	fork,exec,exit, תרגול 3:

תרגול 1

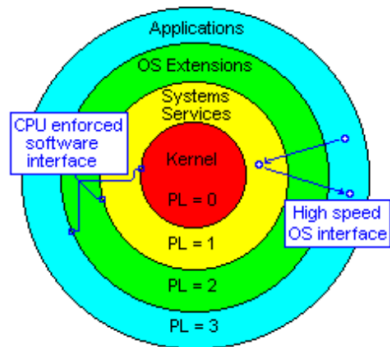
מערכת ההפעלה

מערכת ההפעלה היא **תוכנת מחשב**. התפקיד שלה בעיקר לשמש כמתווכת בין משתמש לחומרה, נותנת שירות לתוכניות ולמשתמשים, מנהלת הרצה של שאר התוכניות ומנהלת את המשאבים של התוכנית (זיכרון, דיסק, מעבד וכו'). ביתר פירוט תפקידיה הם:

- **ניהול זיכרון:** ניהול הקצאה דינאמית של זיכרון כיוון שאלו דברים שמשתנים כל הזמן, מחליטה אילו חלקים של תהליכים לטעון לזיכרון ואילו לא, שומרת מידע על כל פרוסס ומנהלת זיכרון וירטואלי (נדון בהמשך). משתמשת גם בהארד דיסק.
- **ניהול קבצים:** קבצים הינם יחידת אחסון לוגית של מידע, ניהול הרשאות גישה לקבצים, יצירת היררכיה ע"י תיקיות, וכל הפעולות שקשורות לקבצים – יצירה, קריאה, מחיקה וכו'.
- **אחריות כלפי החומרה:** הגנה על החומרה, כיצד? באמצעות מתן גישה מאוד מבוקרת לחומרה. אם היינו יודעים כי רק תוכנית אחת רצה, או שבתוכנית מראש אין שגיאות לא היינו צריכים מערכת הפעלה. אבל אלו כמובן לא הנחות לגיטימיות, בטח לא כיום. כמו כן, **היא צריכה להגן על המעבד:** באמצעות שני מצבי ריצה – *kernel and user modes*. כמו כן, מנגנון הפסיקות (interrupt) מאפשר למערכת ההפעלה להשתלט על המעבד, המעבד עובר לשגרת פסיקה ועובר לקרנל מוד. כמו כן, היא מספקת ממשקים אחידים לחומרה. תפקיד נוסף כלפי החומרה – לנהל וליעל אותה בצורה המרבית (נרצה להשתמש בה כמה שיותר, ונרחיב על כך בהמשך בהרצאות: למשל אנחנו נרצה כי היא תשתמש בכמה שפחות זמן מעבד).

ארכיטקטורת פון נוימן:

איך מחשב פועל?



- מעבד (CPU): אחראי לביצוע התוכנית, תפקידו לבצע את הפקודות (שפת מכונה) המאוכסנות בזיכרון של המחשב. נשאלת השאלה – האם הוא יבצע כל פקודה שנבקש? זה תלוי במושג שנקרא "רמת הפריווילגיה" (Rings) שניתנה לפקודה הנוכחית. כאשר קוד מסוים מבקש לגשת לאזור בזיכרון, החומרה מאשרת את הגישה בהתאם במגבלות הבאות: לפי סוג הפקודה, ולפי *privilege* של המבקש (לכל אזור בזיכרון מוגדר DPL), הגישה מותרת רק אם הפריווילגיה של המבקש קטנה מה DPL. לצורך המחשה (משמאל): אם הקרנל שלנו רוצה להריץ *word*, ה *PL* של הקרנל הוא 0 ושל *word* הוא 3 וכיוון ש $3 > 0$ אזי הוא יכול לבצע.

בכל רגע נתון תוכנית אחת בלבד רצה על המעבד. פסיקה (interrupt) גורמת למעבד להפסיק את ריצתו והשליטה עוברת למערכת ההפעלה – היא נשלחת כשרוצים לבצע קריאת מערכת, שיש שגיאה וכו'.

- זיכרון: מאחסן את הפקודות לביצוע והנתונים שהתוכנית משתמשת בהם.
- רגיסטרים – תאי אחסון נתונים אשר כל הפעולות האריתמטיות והלוגיות מתבצעות עליהם.

PSW: מדובר ברגיסטר שאחד מתפקידיו הוא להבחין אם אנחנו ב *User or kernel mode*, אם ביט מסוים באוגר זה דולק אזי התהליך שרץ חוסם פעולות מסוימות כמו IO, גישה לכל חלקי הזיכרון ועדכון טבלאות/רגיסטרים מסוימים. אם הביט כבוי – יש גישה להכל.

חשוב לשים לב ולזכור: ככל שה *Ring* קטן יותר – רמת ההרשאה גבוהה יותר.

System call: איך תהליך בעל *privilege* גבוה יכול לבצע פעולות שדורשות *privilege* נמוך? ע"י *system call*: מבקשים ממערכת ההפעלה שתסייע. מערכת ההפעלה דואגת לטפל בבקשה ולהחזיר את הנתונים המבוקשים למבקש הבקשה.

Linux ופקודות שימושיות בלינוקס

לינוקס היא מערכת הפעלה שנבנתה בהשראת מערכת Unix. היא חופשית וחינמית, ניתנת להורדה בחינם מהאינטרנט. ודוגלת בגישה של Open Source: קוד פתוח, הקוד פתוח לכולם ומפותח בשיתוף פעולה ע"י אלפי מתכנתים ברחבי העולם (זהו פרויקט בכלל, שהחל ב-1991 ע"י סטודנט), כלומר כל אחד יכול לראות את קוד המקור, לקרוא אותו ולראות בדיוק מה התוכנה עושה. המערכת רצה על כמעט כל סוג מעבד, מאפשרת למספר משתמשים להיות מחוברים למערכת בו זמנית, ותומכת בריבוי תהליכים.

פקודות שימושיות:

- **Man:** פקודה להצגת דפי הסבר על פקודות המערכת השונות. כאשר נרצה לדעת מה פקודה מסוימת עושה ואילו דגלים יש לה נכתוב `man` ואז את שם הפקודה. למשל: `man ls` יציג את דף ההסבר של הפקודה `ls`, ונוכל לראות מה עושים דגלים כמו `-a` וכו'. כמו כן, לפעמים לא נזכור את שם הפקודה המדויק, אך יודע מה אתה רוצה לעשות. לכן ניתן לחפש באמצעותה לפי נושא. כך: `<topic> -k man`, הדגל `-k` מבצע חיפוש במאגר המדריכים.
- דפי המדריך אינם סתם רשימה אחת ארוכה, אלא ספרייה שמחולקת לפי נושאים. זה עוזר לנו לדעת אם אנחנו מסתכלים על פקודת טרמינל או פקודת C. לכן, ישנם סקשנים שונים:
 1. `Section 1-commands`: פקודות משתמש רגילות כמו `cd`, `ls` וכו'.
 2. `Section 2- system calls`: רשימה של קריאות המערכת
 3. `Section 3- C library functions`: פונקציה של ספריות סטנדרטיות כמו `printf`, `scanf` וכו'.ויש עוד... ישנם סה"כ 9 סקשנים.
- **Ls (list directory contents):** פקודה זו מדפיסה את התוכן של ספרייה מסוימת. במידה ונריץ את הפקודה ללא פרמטרים, היא תריץ את תוכנה של הספרייה הנוכחית. אם נוסיף דגלים כמו `a`, למשל `ls -al` יודפסו גם שמות הקבצים הנסתרים ונקבל פלט מורחב עם נתונים נוספים על הקבצים ותתי הספריות.
- **Pwd:** `print name of current/working directory`, מדפיסה את נתיב הספרייה הנוכחית.
- **Cd:** משמשת כמעבר מספרייה לספרייה, ישנן הרחבות: למשל `cd ..` היא מעבר מהספרייה הנוכחית לזו שמכילה אותה, `cd -` כחזרה לספרייה הקודמת שהיינו בה, `cd ~` מעבר לספריית הבית של המשתמש.
- **Mkdir:** יצירת ספרייה חדשה.
- **Rmdir:** מחיקת ספרייה (ריקה!), אם לא ריקה יהיה שגיאה.
- **Mv:** העברת קובץ. למשל: `mv myfile backups/` למעבר הקובץ `myfile` אל הספרייה `backups`.
- **Cp:** פקודה שמעתיקה קבצים, דוגמאות:
העתקת הקובץ `myfile` לתוך הספרייה `backups`: `cp myfile backup/`
העתקת כל הקבצים בעלי הסיומת `txt` לתוך הספרייה `txts`: `cp *.txt /home/op/txts`
- **Chmod:** שינוי הרשאות לקובץ. למשל: שינוי הרשאות לקובץ כך שכל מי שנוגע יוכל להריץ, לקרוא ולכתוב: `chmod 777 myfile`.
- **Cat:** פקודה שמשרשרת תוכן של קבצים ומדפיסה אותם למסך, למשל: הדפסת הקובץ `myfile` למסך ע"י `cat myfile`. ניתן גם להדפיס קובץ אל תוך קובץ אחר: `cat myfile > yourfile`
- **Gcc:** קומפיילר עבור השפות C, C++. למשל קמפול הקובץ: `gcc myprog.c`, שם קובץ ההרצה שיווצר יהיה `a.out`. כמו כן ניתן גם לקמפל ולתת את שם קובץ ההרצה בעצמנו ע"י `gcc myprog.c -o myprog`.

• הרשאות, אבטחה וקבוצות בLinux

לינוקס היא מערכת Multi-user, כלומר כמה משתמשים יכולים להיות מחוברים למערכת במקביל (או בזמנים שונים). לכל משתמש יש userID, מס' ייחודי משלו. באמצעות הפקודה who ניתן לראות מיהם המשתמשים שמחוברים כרגע, כולל התאריך שבו התחבר, מהיכן הוא מחובר וכו'. לינוקס שומרת את פרטי המשתמשים במערכת באמצעות קובץ בשם etc/passwd בו כל שורה בקובץ מתארת משתמש אחד, ומחולקת ל 7 שדות שמופרדים בנקודתיים: שם המשתמש, סיסמה (מסומנת באיקס בפועל, היא מאובטחת ושמורה בקובץ אחר), USERID, GROUPID, וכו'.

קבוצות: כל משתמש יכול להיות שייך למספר קבוצות (יש לו קבוצת ברירת מחדל אליה שייך), כדי לבדוק באילו קבוצות נמצא המשתמש מסוים נוכל להריץ `groups <username>`. משתמש יכול להחליף גם את קבוצת ברירת המחדל שלו ע"י שימוש בפקודה `newgrp <groupName>`. הקובץ `etc/group` מכיל רשימה של כל הקבוצות במערכת. לכל קבוצה נשמרת רשומה בקובץ עם הפרטים הבאים: שם קבוצה, סיסמה, ID, משתמשים שבקבוצה.

מדוע שנשתמש בקבוצות? זו הדרך של המערכת לנהל הרשאות בצורה יעילה – במקום לתת לכל משתמש הרשאות באופן פרטני, היא נותנת פר קבוצה וכל קבוצה מקבלת מפתח למשאבים מסוימים.

ישנן 3 סוגי הרשאות: `read,write,execute`. ההרשאות מסומנות ע"י מספר `execute=1,read=4,write=2`. פירוש ההרשאה 7 למשל: $4+2+1$, כלומר הרשאות כתיבה, קריאה וביצוע.

לכל קובץ או תיקייה במערכת יש כרטיסיית הרשאות שמתייחסת לשלושת הישויות הללו:

- **User:** בעל הקובץ. זהו המשתמש שיצר את הקובץ, ולו לרוב יש את השליטה המלאה – יכול לקרוא לכתוב ולבצע. הוא היחיד פרט לroot שיכול לשנות את ההרשאות של הקובץ לאחרים.
- **Group:** הקבוצה המשויתת. לכל קובץ משויכת קבוצה אחת מסוימת, כל מי שחבר בקבוצה זו יקבל את ההרשאות שהוגדרו עבורו.
- **Other:** כל היתר, כל משתמש במערכת שלא נכנס תחת 2 ההגדרות הקודמות. ההרשאות כאן יהיו הכי מצומצמות.

User Mask: מדובר במסכה שחלה על תהליכים במערכת, כשתוכנה יוצרת קובץ חדש `umask` מגביל את ההרשאות שלו באופן אוטומטי על מנת למנוע מצב שקובץ נוצר עם הרשאות רחבות מדי. הפקודה מחזירה מספר שמסמל את הmask של התהליך שהריץ את הפקודה.

כיצד משנים סיסמה? על מנת לשנות סיסמה, המערכת חייבת לכתוב את הסיסמה החדשה לתוך הקובץ של המערכת שכפי שאמרנו `etc/passwd/`. הקובץ הנ"ל נמצא בבעלות root ואין למשתמש רגיל הרשאות של כתיבה אליו. אז איך הסיסמה תעודכן?

בהרשאות ישנה האות s, זוהי הרשאה מיוחדת שנקראת SetUID. כל מי שיש לו הרשאת גישה על הקובץ (למשל, המשתמש הרגיל) יריץ את התוכנה לא עם ההרשאות שלו, אלא עם ההרשאות של יוצר התוכנה (במקרה של הסיסמה היוצר הוא root). זה מה שמאפשר לקבל כוחות של root לזמן קצר שהפקודה הזו רצה. כיצד קובעים הרשאה זו? באמצעות `chmod` ומוסיפים ספרה רביעית לפני שלוש הספרות הרגילות, כאשר הספרה 4 קובעת את s עבור uid (משתמש), הספרה 2 עבור הקבוצה והספרה 6 עבור שניהם.

תרגול 2

מהו shell? מדובר בתוכנית שמאפשרת ממשק לפקודות בין משתמש יחיד לבין מערכת ההפעלה (kernel). ישנן 2 משפחות של shell:

- Sh: סינטקס שמזכיר פסקל, הגרסה הכי פופולרית היא bash.
- Csh: סינטקס שמזכיר C, הגרסה הכי פופולרית היא tcsh.

ניתן לבדוק מהו shell שאנו משתמשים בו באמצעות הפקודות `echo $SHELL`.

בכל תהליך, קיימים **שלושה** ערוצים סטנדרטיים פתוחים שדרכם עובר המידע. כל ערוץ שכזה מיוצג ע"י `fd`:

1. `Fd 0 (stdin)`: ערוץ קלט סטנדרטי, משם התוכנית קוראת נתונים.
2. `Fd 1 (stdout)`: ערוץ פלט סטנדרטי, לשם התוכנית כותבת.
3. `Fd 2 (stderr)`: ערוץ שגיאות סטנדרטי, הוא מחובר למסך כברירת מחדל, הוא נפרד למען הודעות שגיאה דחופות.

על פקודות ושכדאי לדעת:

- אם נעשה `ls > a.txt` (באשר `a.txt` הוא קובץ כלשהו, נכתוב לתוכו למעשה את שמות כל הקבצים בתיקייה הנוכחית). – אם הקובץ הנ"ל `a.txt` לא היה קיים, shell יצר אותו לאחר פקודה זו והכניס אליו את שמות הקבצים.
- `>`: הפניית פלט עם דריסה. למשל, אם נעשה `a.txt > echo "Hello!"` אז לא משנה מה היה קודם בקובץ `a.txt` כעת יהיה בו "Hello" בלבד.
- `>>`: הפניית פלט ללא דריסה, עם הוספה.
- `<`: הפניית קלט מקובץ לפקודה. למשל: `sort < a.txt` ייקח את כל הערכים שבקובץ ויעביר אותם אל פונקציית `sort` שתמיין אותם.
- `<<`: מאפשר כמה שורות קלט שנרצה לכתוב עד שנגיע ל`delimiter`. דוגמה:

```
Bye!
yaaron@GuyHP17:~$ << Guy
> Hey!
> Bye!
> Guy
```

- `Wc`: מדובר בפקודה שעוזרת לניתוח קבצים, אם נרץ סתם `a.txtwc` נקבל כברירת מחדל: כמות שורות, כמות מילים, כמות בתים. ישנם דגלים שמאפשרים לקבל רק נתון ספציפי: `-c` למס' בתים, `-m` למס' תווים, `-w` למס' מילים ו-`-l` למס' שורות (סופר כמה פעמים יש `\n`). נשים לב כי בסיום המס' שורות/תווים/מילים וכו' יופיע גם באותה שורה שם הקובץ.
- הכוח של פקודה זו מגיע משילובה עם פקודות אחרות, למשל `ls |wc` ידפיס כמה קבצים יש בתיקייה הנוכחית.
- `Head`: כברירת מחדל מדפיסה 10 שורות ראשונות של הקובץ/קבצים שצוינו. אם נעשה `num`-יודפסו השורות הראשונות בקובץ.
- `Tail`: כברירת מחדל מדפיסה 10 שורות אחרונות של הקובץ/קבצים שצוינו. אם נעשה `num`-יודפסו השורות האחרונות בקובץ.
- `Pipeline connection`: מקרה של הפניית קלט פלט, פלט של תוכנית אחת הופך לקלט של התוכנית הבאה. למשל `who |wc` יציין את מס' המשתמשים המחוברים. (אין מגבלה על כמה פעמים ששמים |).
- משתנים: **משתנים מקומיים** בשל מוכרים רק לשל, לא לסקריפט/פקודות אחרות. כך נאתחל משתנה: `NEWVAR=" Hey"`

משתני סביבה

כאשר נכנסים shell נקבעת סביבת עבודה, שכוללת את ספריית הבית, סוג הטרמינל עליו עובדים וכו'. ערכים אלו נשמרים בתוך משתני סביבה. משתני סביבה מועברים בין התוכניות והפקודות ולא צריך להגדיר משתנה לפני השימוש בו. בשביל להפוך משתנה רגיל למשתנה סביבה נשתמש בexport.

כדי לשנות ערך של משתנה סביבה, נרשום כך: שם המשתנה בצד שמאל ללא דולר, שווה, לערך החדש בצד ימין. **הערך ישתנה רק עבור הטרמינל הנוכחי והתוכניות שהוא יריץ**, על מנת לשנות ערך משתנה סביבה עבור המערכת כולה יש לערוך קובץ בשם etc/environment. על מנת לראות את כל משתני הסביבה, נוכל להשתמש בפקודה env.

סקריפט (script)

סקריפט הוא תוכנית שמורצת ע"י מפרש (interpreter) ללא קומפילציה. המפענח מפרש את הקוד שורה שורה ומבצע אותה. מפרש נפוץ בהקשר של shells הוא bash. יתרונות – מהירות, יכול עיבוד מחרוזות טובה.

כיצד כותבים סקריפט?

1. נפתח קובץ טקסט, ונוסיף בשורה הראשונה את Shabang: #!/bin/bash (על מנת שהמערכת תדע איזה מפרש יש להריץ).
2. נכתוב את הפקודות שנרצה. (לא צריך סיומת מיוחדת אם כי נהוג ונחמד לשים exit 0).
3. בסוף נריץ.

ארגומנטים לסקריפט: נרצה להעביר לסקריפט ארגומנטים, איך זה עובד? השל לוקח את המילים שכתבנו בשורת הפקודה אחרי שם הסקריפט ומכניס אותן לתוך משתנים מיוחדים - \$0 מכיל את שם הסקריפט עצמו, \$1 את הארגומנט הראשון ששלחנו וכך הלאה... כלומר כך נשלח אותם:

Guy ./myscript.sh

איך נוכל להריץ סקריפטים מכל מקום? נרצה להפוך אותו לפקודה שמוכרת ע"י המערכת.

בשלב ראשון: ניצור תיקייה בתוך תיקיית הבית שתשמור את כל הסקריפטים שלנו: mkdir \$HOME/myScripts

בשלב שני: נעביר את הסקריפט שכתבנו לתוך התיקייה הנ"ל: mv dirFiles \$HOME/myScripts

בשלב שלישי: נרצה לעדכן את משתנה הסביבה PATH: PATH=\$HOME/myScripts:\$PATH (מה קורה כאן בעצם? אומרים לשל: כשאתה מחפש פקודות, חפש קודם בתיקיית הסקריפטים שלי ואז בשאר המקומות הרגילים).

כעת, אם נקליד dirFiles מכל מקום בטרמינל, הוא ירוץ כמו פקודה רגילה. **חשוב:** אם נסגור את הטרמינל, השל ישכח את הניתוב הזה.

עוד פקודות וסינטקס:

- Grep: הפקודה מחפשת תבנית טקסט מסוימת בתוך קבצים, ומדפיסה רק את השורות שמכילות תבנית זו. כך סינטקטית: Grep [options] pattern [files]. אפשר להוסיף דגלים: -c למס' השורות שמכילות את התבנית, -i השורות שתואמות לתבנית.

סיכום מערכות הפעלה | © גיא יער-און

- ביטויים אריתמטיים: על מנת שהשל יבין שמדובר בחישוב אריתמטי נשתמש בסוגריים מרובעים. אם נכתוב `num1=$num1+7` התוצאה תהיה כמחרוזת, לכן אם `num1=8` התוצאה תהיה "7+8". לעומת זאת, `num1=${num1+7}` יחשב כמו שצריך.
- הפקודה `test`: הפקודה מחזירה אמת או שקר עבור ביטוי מסוים. כאשר נשים לב כי זה הפוך ממה שאנחנו חושבים לרוב: אמת = 0, שקר = 1.

```
yaaron@GuyHP17:~$ test 1=2
yaaron@GuyHP17:~$ echo $?
0
yaaron@GuyHP17:~$ test 1 = 2
yaaron@GuyHP17:~$ echo $?
1
```

דוגמה: בשורה הראשונה אנחנו מבצעים השמה – ללא הרווח אנחנו פשוט מבצעים השמה, שהצליחה, לכן חוזר 0 ("אמת"), בדוגמה השנייה אנחנו מבצעים בדיקה (יש רווח), לכן חוזר הפעם באמת 1 ("שקר"). כמו כן, `test` זו תוצאת חישוב הטסט האחרון.

- **מערך:** נגדיר מערך באופן הבא – `arr=(x1 x2 x3)` וכו'. נוכל לבצע השמת משתנים באופן הבא: `arr[0]="Hey!"` וכו'. ניגש לאיבר באופן הבא: `echo ${arr[8]}` נגש לכל האיברים בצורה הבאה: `echo ${arr[@]}` או `echo ${arr[*]}`. מה ההבדל? עם `@` ירידת שורה בין כל ערך, ועם `*` בשורה אחת.

Flow Control

נציג כאן סינטקסית מה עלינו לזכור:

```
if condition_command :lf .1
then
  true_commands
else
  false_commands
fi
```

זה הפוך מבדרך כלל, אם `condition command=0` יבוצע `true commends`.

```
while conditions
do
  commands
done :While .2
```

```
for variable in wordlist
do
  commands
done :For .3
```

```
function_name () {
  commands
} :4 פונקציות
```

```
#!/bin/bash

# Define the function
greet() {
  echo "Hello, $1!"
}

# Call the function with an argument
greet "World"
```

לדוגמה:

תרגול 3

- PID: מספר ייחודי שמתקבל בזמן יצירת התהליך.
- UID: מספר ייחודי של המשתמש שמריץ את התהליך.
- PPID: המזהה של התהליך שיצר את התהליך הנוכחי.
- Mode: מצב התהליך (נוצר, ממתין, מחכה וכו').
- Priority: מספר שמסמל את עדיפות הרצת התהליך.

כיצד נוצר תהליך? כאשר מערכת ההפעלה נטענת נוצרים תמיד 2 תהליכים:

1. Pid=0, swapper: אחראי על ניהול הזיכרון.
2. Pid=1, init: כל התהליכים הופכים להיות צאצאים שלו.

Fork()

בלינוקס, על מנת ליצור תהליכים משתמשים בפקודה `fork()`. התהליך המקורי נקרא **תהליך אב**, והתהליך שנוצר נקרא **תהליך בן**. הפעולה משכפלת את תהליך האב לתהליך הבן, וממשיכה לשורת הקוד הבאה בשני התהליכים: התהליך המקורי שקרה לפעולה, והתהליך החדש שנוצר בעקבות הרצת הפעולה. לתהליכים יש:

- קוד זהה (נמצאים באותה נקודה בתוכנית, מיד לאחר הקריאה ל`fork()`!)
- זיכרון זהה (משתנים וערכיהם)
- סביבה זהה (קבצים פתוחים, `fd` וכו')
- PID שונה!

על מנת להשתמש בפקודה נבצע `#include <unistd.h>`, `#include <sys/types.h>` ונבצע כך: `pid_t fork()`.

ערך מוחזר: במקרה של כישלון, מוחזר -1 לאב. אחרת: לבן מוחזר 0 ולאב מוחזר `pid` של הבן (`pid > 0`) ולכן זו דרך להבדיל בניהם (בקוד).

- לאחר פעולת `fork()` מוצלחת, לאב ולבן ישנם אותם משתנים בזיכרון אך בעותקים נפרדים. כלומר: שינוי ערכי המשתנים אצל האב לא יראה אותו דבר אצל הבן ולהפך. תהליך הבן הוא תהליך נפרד לחלוטין מתהליך האב!

מנגנון copy on write: ייחודי ללינוקס, מאפשר להשתמש בדפים משותפים לשני התהליכים עד לנקודת הזמן הראשונה בה אחד מן התהליכים מבקש לשנות דף. ואז – מערכת ההפעלה מעתיקה את הדף הזה כך שלכל תהליך יהיה עותק פרטי משלו, ורק אז מאפשרת לתהליך לכתוב על הדף. כלומר: **התהליך נוצר בפועל רק כאשר ישנו שינוי בניהם.**

שאלה. נתבונן בקטע הקוד הבא: מה יודפס?

תשובה. לא ניתן לדעת! ייתכן כי האבא ידפיס קודם, יתכן כי הבן קודם. הדבר היחידי שלא יתכן: 3 יודפס בתחילה. לכן ייתכן:

- 1323 (קודם בן, אח"כ אבא)
- 2313 (קודם אבא, אח"כ בן)
- 2133 (אבא, בן, ואז המשך הקוד)
- 1233 (בן, אבא, ואז המשך הקוד)

```
void main()
{
    pid_t pid;
    if ((pid = fork()) == 0)
        printf("1");
    else
        printf("2");
    printf("3");
}
```


שאלה. נניח ונריץ את קטע הקוד הבא:

Fork()

Fork()

...

Fork()

כמה תהליכים ייווצרו?

תשובה: נסמן את מס' הפעמים שנקראה הפונקציה בח. אזי, כל תהליך פותח 2 חדשים כמעין עץ בינארי מלא. לכן מס' התהליכים כמס' העלים 2^n . אם נרצה לספור את מס' התהליכים החדשים שיווצרו אזי מדובר ב- $2^n - 1$.

- כאשר אב מסיים את פעולתו והבן עדיין חי, הבן נחשב ליתום – orphan. התהליך ($\text{init}(\text{pid}=1)$) מאמץ דיפולטיבית את כל היתומים (כלומר עבורם $\text{PPID}=1$).

ישנן מספר פונקציות שימושיות בהקשר של `fork()`:

- `Pid_t getpid()`: מחזיר מזהה של תהליך הקורא.
- `Pid_t getppid()`: מחזיר את מזהה תהליך האב של התהליך הקורא.
- `Uid_t getuid()`: מחזיר את מזהה המשתמש של התהליך הקורא.

```
pid_t pid;
pid = getpid();
while (fork() == 0) {
    if (pid == getpid())
        break;
}
```

שאלה. נתבונן בקטע הקוד הבא:

כמה תהליכים ייווצרו בעקבות ביצוע הקוד?

פתרון: ייווצרו אינסוף תהליכים. מדוע? התהליך המקורי קורא ל-`fork()`, יוצר בן ומסיים (כי הערך שחוזר אצלו שונה מאפס). תהליך הבן, נכנס לתוך הלולאה ואצלו לא מתקיים התנאי `pid==getpid()`, כיוון שמזהה תהליך שלו שונה משל האב. לכן נוצר תהליך נכד ותהליך הבן מסתיים (שוב, כי הפעם אצלו `fork()!=0`). למעשה, בכל פעם "יוצרים" ילד חדש, שעבורו לא מתקיים התנאי שבתוך הלולאה ולכן אין ברייק, והתהליך הקודם מסיים את ריצתו ב-`while()` הבא. לכן, נוצר מעין עץ בינארי בצורת "שרוך" אינסופי של היררכיית ילד-אב.

Exec()

נניח כי נרצה להריץ תוכנית מסוימת מתוך תהליך כלשהו, נוכל ליצור תהליך נוסף ע"י פורק, אבל התהליך הזה יהיה זהה לתהליך שממנו נעשתה הקריאה. `Exec` טוענת תוכנית חדשה לביצוע במקום התוכנית שהייתה אמורה להתבצע. להלן הסינטקס:

```
#include <unistd.h>
int execl(char *pathname, char *arg0, ..., NULL)
```

- פרמטר ראשון הוא שם הקובץ שמכיל את התוכנית החדשה לביצוע, החל מן הפרמטר השני: מצביעים למחרוזות שמכילים את הפרמטרים עבור התוכנית הנקראת. באופן מוזר – הפרמטר הראשון חייב להיות שם הקובץ (כך למעשה נעביר לפונקציה פעמיים את אותו איבר), והאחרון חייב להיות `NULL`.

סיכום מערכות הפעלה | © גיא יער-און

- ערך מוחזר: אם יש כישלון יוחזר -1, אחרת: לא נתעניין בערך המוחזר כי הקריאה כלל אינה חוזרת כי exec החליפה את התוכנית הנוכחית בחדשה.

קיימים מספר וריאנטים לפונקציה: `exec(l,v){optional: e,p}`:

- L: הארגומנטים מועברים כרשימה של מחרוזות המופרדים בפסיקים ומסתיימת בNULL.
- V: הארגומנטים מועברים כמערך של מצביעים `argv*` כאשר האיבר האחרון במערך הוא NULL.
- E: מאפשר להעביר באופן מפורש מערך של משתני סביבה לתהליך החדש.
- P: מדובר במשתנה הסביבה PATH, כך שיעזור לאתר את קובץ ההרצה, ולא נצטרך לתת נתיב מלא של קובץ התוכנית החדשה.

לדוגמה:

```
char *args[] = {"/bin/ls", "-r", "-t", "-l", NULL};  
execv("/bin/ls", args);
```

```
char *args[] = {"ls", "-r", "-t", "-l", NULL};  
execvp("ls", args);
```

:Wait()

כאשר הילד רץ, ההורה: ממשיך לבצע את פעולותיו + ממתין לילד שיסיים. כך נראה סינטקס הפקודה:

```
#include <sys/types.h>  
#include <sys/wait>  
pid_t wait(int *status)
```

- בקריאה לפונקציה, התהליך הקורא (האב) ימתין עד שאחד מבניו יסתיים.
- הערך המוחזר הוא המזהה של התהליך בן שתהליך הקורא ממתין לו.
- הפרמטר status יכיל מספר שניתן יהיה לחלץ ממנו מידע על סטטוס תהליך הבן שהתהליך הקורא ממתין לו.

:Context switch

תהליך יחיד ראשון init נוצר ומקבל PID=1. במהלך ריצתו, נניח והוא יוצר תהליך נוסף שעתיד להתחרות בו על משאבי המערכת. בה"כ PID=2. מערכת ההפעלה תחליף בין PID=1 וכל הנתונים הנלווים לו לבין PID=2 תוך שמירה על חוצץ בין התהליכים. לפני שהתהליך מורדם – נתוניו נשמרים ולפני שהוא "מתעורר" נטענים אותם נתונים לאותם מקומות בדיוק. בשעת ריצת התהליך, הוא רוצה את כל משאבי המערכת הנחוצים לריצתו. אם משאב נחוץ תפוס ע"י תהליך אחר הוא יורדם שוב עד לתור הבא לנסיונו.

:Waitpid()

פונקציה אשר משהה את ביצוע התהליך הנוכחי עד שתהליך בן ספציפי משנה את מצבו – מאפשרת שליטה גמישה על אופן ההמתנה באמצעות options שנבחרו. סינטקטית:

```
pid_t waitpid(pid_t pid, int *status, int options)
```

:exit()

פונקציית ספרייה שקוראת לסיים את התהליך באופן מיידי. Flush הוא הפעולה שבה המחשב מעביר נתונים שנשמרו בזיכרון זמני אל היעד הסופי שלהם (כמו דיסק). ישנן 2 פונקציות רלוונטיות בהקשר זה:

סיכום מערכות הפעלה | © גיא יער-און

- `Void_exit(int status)`: סיום התהליך קורא באופן מיידי, אין שום חיוב שיהיה `flush`.
 - `Void exit(int status)`: סיום התהליך, כמעט באופן מיידי, כן יתבצע `flush`.
 - `int atexit(void (*function)(void))`: פונקציה זו "רושמת" פונקציה נתונה להיקרא בסיום נורמלי של תהליך או דרך `return` מה `main` של התוכנית. הפונקציות שמוגדרות נקראות בסדר הפוך לסדר הרישום שלהן – כמו מחסנית. אין ארגומנטים שמועברים, אותה פונקציה יכולה להיות רשומה מס' רב של פעמים והיא תקרא פעם אחת פר רישום, מחזירה 0 אם הצליחה, אחרת ערך ששונה מאפס.
- זומבים: כאשר בן מסתיים והאב לא המתין לו, הבן נחשב כזומבי. ה `PCB` שלו עדיין נשמר – כי האבא עדיין יכול לעשות `wait` – רק אז ה `PCB` יימחק. במידה והאב לא יעשה `wait` לבן הבן יהפוך ליתום, והתהליך `init` יאמץ אותו ויעשה לו `wait()`.

סיפורים של סוף תרגול:

לפי עינת, המתרגלים ביקשו וקיבלו אישור מדודי שתהיה במבחן שאלת בונוס של 3 נקודות על הסיפורים שמופיעים בסוף כל תרגול ולכן הם מרוכזים פה על מנת הנוחות שלנו:

- **תרגול 1:** ב-2015, מרקו מרסאלה פרסם בפורום טכני שהוא בטעות הריץ פקודת `rm -rf` עם הרחבת משתנה שגויה על שרתי חברת האחסון שלו – ומחק את כל הנתונים של 1,535 אתרים, כולל הגיבויים שהיו מעוגנים לאותה מערכת קבצים. העסק נהרס בפקודה אחת. זו הסיבה שחשוב להבין איך ה-`shell` מרחיב משתנים, איך `rm` עובד ברמת ה-`syscall`, ולמה `rm` לא באמת מוחק

סיכום מערכות הפעלה | © גיא יער-און

נתונים (הוא מבצע unlink ל-inode). להכיר את מערכת ההפעלה זה להבין מה פקודה באמת עושה – לא מה שאתם חושבים שהיא עושה. מה המסר? **פקודה אחת שגויה יכולה למחוק חברה שלמה.**

- **תרגול 2:** בספטמבר 2014 התגלה באג בן 25 שנה ב-Bash עצמו – אותו shell שלמדתם היום לכתוב בו סקריפטים. הבאג, שנקרא (CVE-2014-6271) Shellshock, ניצל את האופן שבו Bash מטפל במשתני סביבה והגדרות פונקציות. תוקף יכול היה להזריק פקודות שרירותיות דרך משתני סביבה מעוצבים במיוחד. מכיוון ש-Bash נמצא בכל מקום – שרתי ווב (CGI), לקוחות DHCP – הבאג הזה חשף מאות מיליוני מערכות ברחבי העולם. הבאג התקיים מאז Bash גרסה 1.03 ב-1989. לקח 25 שנה עד שמישהו שם לב. ה-Shell הוא לא רק כלי עבודה, הוא משטח תקיפה קריטי. באג אחד ב-Bash השבית מיליוני מערכות והזכיר לעולם שגם בסקריפטים פשוטים – *With great power comes great responsibility!*.
- **תרגול 3:** ה-fork bomb הקלאסי: `{&:|};`; הוא רק 13 תווים – והוא יכול להפיל כמעט כל מערכת Linux לא מוגנת תוך שניות. זוהי פונקציה שקוראת לעצמה פעמיים, יוצרת צמיחה מעריכית של תהליכים. fork bomb בודד יכול ליצור מעל מיליון תהליכים בפחות מ-60 שניות. בתקרית אמיתית, pull request זדוני ב-GitHub Actions הכיל fork bomb בתוך סוויטת הבדיקות, מה שהפיל את כל ה-runner host והשפיע על builds אחרים. אז מה המסר? גם פקודה שנראית תמימה יכולה להוריד שרתים שלמים ברגע ולכן חשוב ולפעול בזהירות ובצורה בטוחה, מי שמנסה לבלוע יותר מדי תהליכים "במזלג" אחד – עשוי לגרום למערכת כולה להיחנק (או במילים אחרות, תפסת מרובה - לא תפסת).